

Trabajo Práctico Integrador - Arquitectura

Título del trabajo
VIRTUALIZACIÓN

Alumnos

Oliverio Arce- comisión 8 - mail: oliverio.arce97@gmail.com

Mauro Liciaga - comisión 6 - mail: mauroliciaga_lp@hotmail.com

**TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN - UNIVERSIDAD TECNOLÓGICA
NACIONAL.**

Arquitectura y Sistemas Operativos

Docentes:

Nicolas Llanea (comisión 8).

Nicolas Carcaño (comisión 6).

Fecha de entrega: 25/06/2026

TABLA DE CONTENIDOS

• INTRODUCCIÓN	3
• MARCO TEÓRICO	4
• CASO PRÁCTICO	9
• METODOLOGÍA UTILIZADA	18
• RESULTADOS OBTENIDOS	21
• CONCLUSIONES	22
• BIBLIOGRAFÍA	24

INTRODUCCIÓN

En la industria de Tecnologías de la Información moderna, la virtualización es un estándar a la hora de realizar desarrollos. La virtualización permite desplegar sistemas operativos enteros dentro de una misma máquina, con diversos objetivos y utilidades, sin afectar al sistema operativo padre donde estos se alojan. Es posible probar programas en desarrollo y su función en distintos sistemas operativos, ver cómo se comunican entre sí dichos sistemas, etc. Todo ello sin necesidad de costear más hardware del que actualmente se dispone, debido a que todos los recursos de un sistema son virtualizados. Disco duro, sistema operativo, memoria, RAM, procesadores y otros componentes, son factibles de ser emulados mediante software de virtualización. Dicho esto, es de vital importancia que los futuros técnicos en programación incorporen habilidades y competencias para manejar este tipo de software, ya que un programador no solamente escribe código, sino que desarrolla soluciones empresariales en su totalidad. Las cuales deben poder ser implementadas, modificadas y escaladas en diversos sistemas con distintas características. Es por ello que el presente trabajo supone una primera aproximación real de implementación de una máquina virtual en los ordenadores de los autores, para aprender así su funcionalidad de forma práctica, investigando, probando características, equivocándose y aprendiendo de los errores cometidos. Los objetivos propuestos del trabajo fueron implementar un hipervisor de tipo 2 completamente funcional con Ubuntu, un sistema operativo Linux que, al ser software libre, permite adquirirlo sin costo alguno, e implementarlo en una máquina virtual a través del software de virtualización Virtualbox, también libre y gratuito. Una vez desplegado el sistema operativo, se implementó un pequeño script en Python para calcular promedios, y fue probado exitosamente.

MARCO TEÓRICO

La virtualización es una tecnología que permite crear computadoras "virtuales" dentro de una computadora física. Esto hace posible ejecutar varios sistemas operativos en un mismo equipo sin necesidad de tener varias computadoras reales.

Gracias a la virtualización, se pueden aprovechar mejor los recursos del hardware, realizar pruebas de software de forma segura y trabajar con distintos sistemas operativos sin modificar el sistema principal de la computadora.

Actualmente, esta tecnología es muy utilizada tanto en empresas como en entornos educativos, ya que facilita el desarrollo, las pruebas y la administración de sistemas informáticos.

Ventajas:

- Permite utilizar mejor los recursos de la computadora.
- Facilita la realización de pruebas sin afectar al sistema principal.
- Permite trabajar con diferentes sistemas operativos en un mismo equipo.
- Reduce costos al no requerir hardware adicional.
- Facilita la recuperación de sistemas en caso de errores.
- Proporciona entornos aislados y seguros para experimentar.

Máquina virtual (definición):

Una Máquina Virtual es una computadora creada mediante software que funciona dentro de otra computadora. Aunque no existe físicamente, se comporta como si fuera un equipo real.

Una máquina virtual puede tener su propio sistema operativo, memoria, almacenamiento y configuración independiente. Esto permite instalar programas y realizar tareas de forma aislada del sistema principal.

Por ejemplo, una computadora con Windows puede ejecutar una máquina virtual con Ubuntu sin que ambos sistemas interfieran entre sí.

Componentes de una máquina virtual:

Recurso Físico	Recurso Virtual
Procesador	Procesador Virtual
Memoria RAM	Memoria asignada
Disco Rígido	Disco Virtual
Red	Adaptador de red

Hipervisor:

Para que una máquina virtual funcione es necesario un software especial llamado hipervisor, el cual es el encargado de crear y administrar las máquinas virtuales, y asignándoles los recursos como memoria RAM, el espacio en el disco y la capacidad de procesamiento.

Existen dos tipos de hipervisores:

Hipervisor Tipo 1:

Los hipervisores de Tipo 1 funcionan directamente sobre el hardware de la computadora, sin necesidad de un sistema operativo intermedio.

Son los más utilizados en empresas y centros de datos porque ofrecen un mejor rendimiento y una administración más eficiente de los recursos.

Esquema

Hardware —> Hipervisor Tipo 1 —> Máquinas Virtuales —> Sistemas Operativos

Hipervisor Tipo 2:

Los hipervisores de Tipo 2 funcionan como un programa más dentro de un sistema operativo ya instalado.

Por ejemplo, si una computadora tiene Windows instalado, se puede instalar VirtualBox y desde allí crear máquinas virtuales con Ubuntu, Fedora, u otros sistemas operativos.

Este tipo de virtualización es muy utilizada para estudiar, desarrollar software y realizar pruebas.

Esquema

Hardware —> Sistema Operativo (Windows, Linux, etc.) —> Hipervisor Tipo 2 —> Máquinas Virtuales —> Sistemas Operativos Invitados

En este trabajo se utilizará un hipervisor de Tipo 2, ya que la máquina virtual será creada mediante VirtualBox sobre una computadora que ya cuenta con un sistema operativo instalado.

Ubuntu

Ubuntu es una distribución del sistema operativo Linux desarrollada por Canonical. Es uno de los sistemas operativos más utilizados dentro del mundo del software libre debido a su estabilidad, seguridad y facilidad de uso.

Se utiliza tanto en computadoras personales como en servidores y es una opción muy elegida para programación, administración de sistemas y entornos de desarrollo.

Entre sus principales características se encuentran:

- Es gratuito y de código abierto.
- Tiene una gran comunidad de usuarios.
- Recibe actualizaciones periódicas.
- Es compatible con una amplia variedad de herramientas de desarrollo.
- Posee una interfaz amigable para usuarios principiantes y avanzados.

Ejecución de programas en una máquina virtual

Uno de los usos más comunes de las máquinas virtuales es la posibilidad de crear entornos de prueba para ejecutar aplicaciones sin afectar el sistema operativo principal.

En este trabajo se creará una máquina virtual utilizando VirtualBox, se instalará Ubuntu como sistema operativo invitado y posteriormente se ejecutará un programa dentro de dicho entorno.

Esto permitirá comprobar el funcionamiento del código en un sistema Linux y comprender cómo se utilizan las máquinas virtuales en situaciones reales de desarrollo y pruebas de software.

Importancia de la virtualización para un Técnico en Programación

La virtualización es una herramienta muy importante para la formación de un Técnico en Programación porque permite trabajar con distintos sistemas operativos, realizar pruebas de programas de forma segura y aprender a administrar entornos similares a los utilizados en empresas.

Además, brinda experiencia práctica en el uso de herramientas ampliamente utilizadas en el ámbito profesional, facilitando la comprensión de conceptos relacionados con sistemas operativos, redes, servidores y despliegue de aplicaciones.

Por estos motivos, conocer y utilizar tecnologías de virtualización constituye una habilidad valiosa para cualquier profesional vinculado al desarrollo de software.

CASO PRÁCTICO

En primer lugar, se procede a explicar el resumen de lo que se hizo, y luego se mostrará el detalle de los pasos realizados. Se utilizó VirtualBox para instalar una máquina virtual con Ubuntu Desktop. Se instaló dicho sistema operativo en una máquina virtual montada en VirtualBox. Paralelamente, se creó en el sistema operativo anfitrión un pequeño script en Python para calcular promedios, y se utilizó la terminal de Ubuntu para ejecutar los comandos en Git y clonar el repositorio en Github al cual se había subido dicho script en Python. Finalmente, con una copia local del repositorio en Ubuntu, se ejecutó el código para analizar su correcto funcionamiento. Es importante destacar que para todos los casos mencionados, ambos integrantes del equipo realizaron las mismas pruebas, con el objeto de implementar la misma serie de pasos y realizar las pruebas correspondientes, lo cual tiene como fin en primer lugar el aprendizaje que dicha implementación conlleva, y por otro lado probar que el código funcione en dos máquinas virtuales distintas, garantizando su escalabilidad.

1. Descargando y configurando VirtualBox

El primer paso en el proyecto fue descargar VirtualBox, un hipervisor de código abierto en el cual se realizaron las pruebas posteriormente. Para ello, se ingresó a la pagina <https://www.virtualbox.org/wiki/Downloads> , y desde allí se descargó la versión para Windows, y su correspondiente paquete de extensión, siguiendo con los lineamientos planteados por la cátedra de Arquitectura y Sistemas Operativos. Una vez realizada la descarga, se procedió a la instalación del VirtualBox siguiendo las instrucciones del programa.

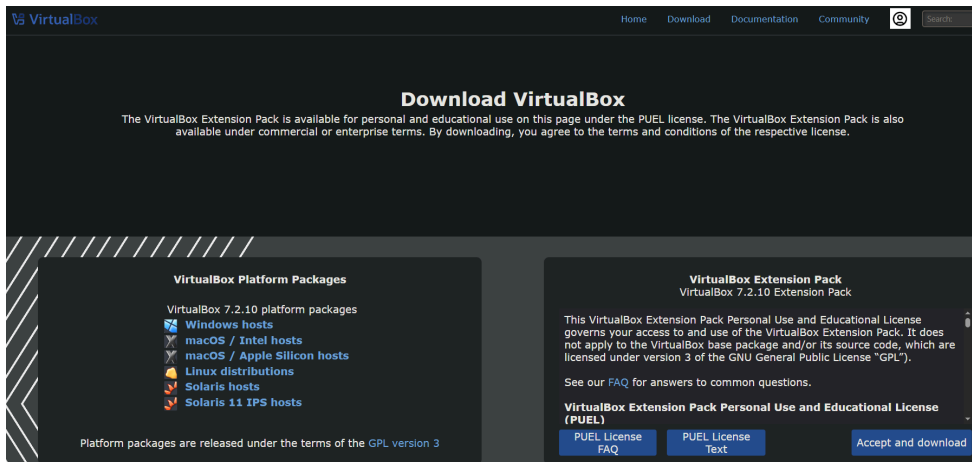


Figura 1. Página de descargas de VirtualBox

2. Implementando la máquina Virtual con Ubuntu

Una vez la instalación estuvo completa, el siguiente paso fue configurar la máquina virtual para realizar las pruebas correspondientes. El primer paso dentro de esto fue descargar el sistema operativo Ubuntu 26.04 LTS. Este sistema operativo es de código abierto, gratuito y relativamente liviano, lo cual lo hace ideal para pruebas de virtualización.

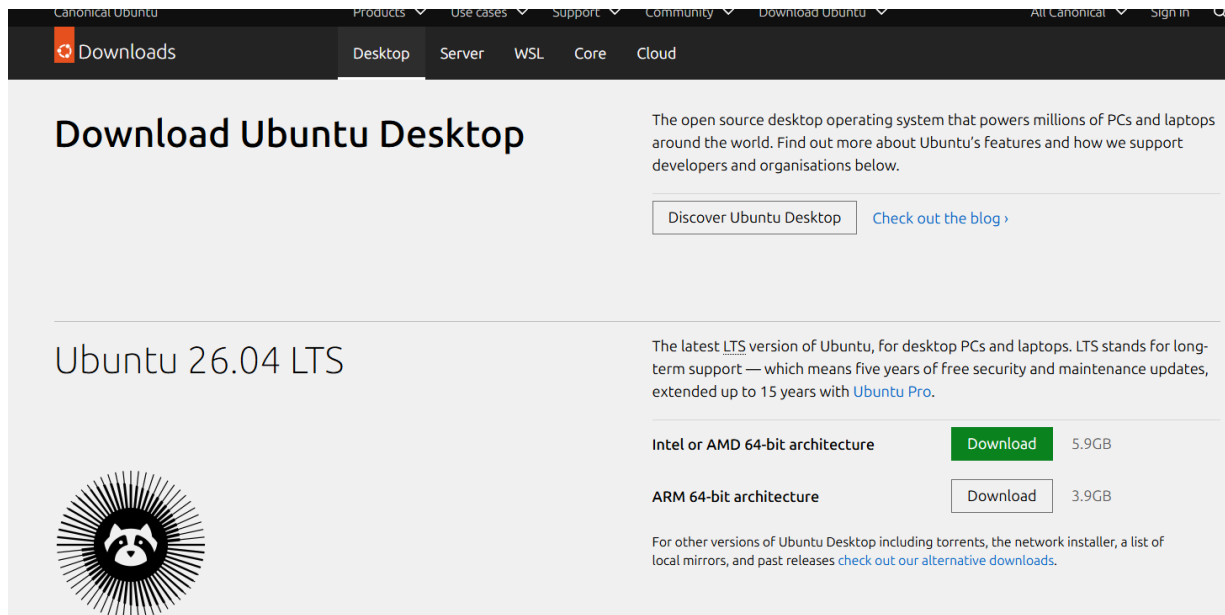


Figura 2. Página de descargas de Ubuntu

De allí se descargó el archivo ISO del Sistema Operativo, el cual fue utilizado para montar en la máquina virtual y proceder a la instalación de Ubuntu.

Una vez finalizada la descarga, el equipo realizó la implementación de la máquina virtual. Para esto se deben configurar una serie de parámetros. En primer lugar, ponerle nombre a la VM, seleccionar la carpeta donde será guardada, y seleccionar el archivo ISO del SO.

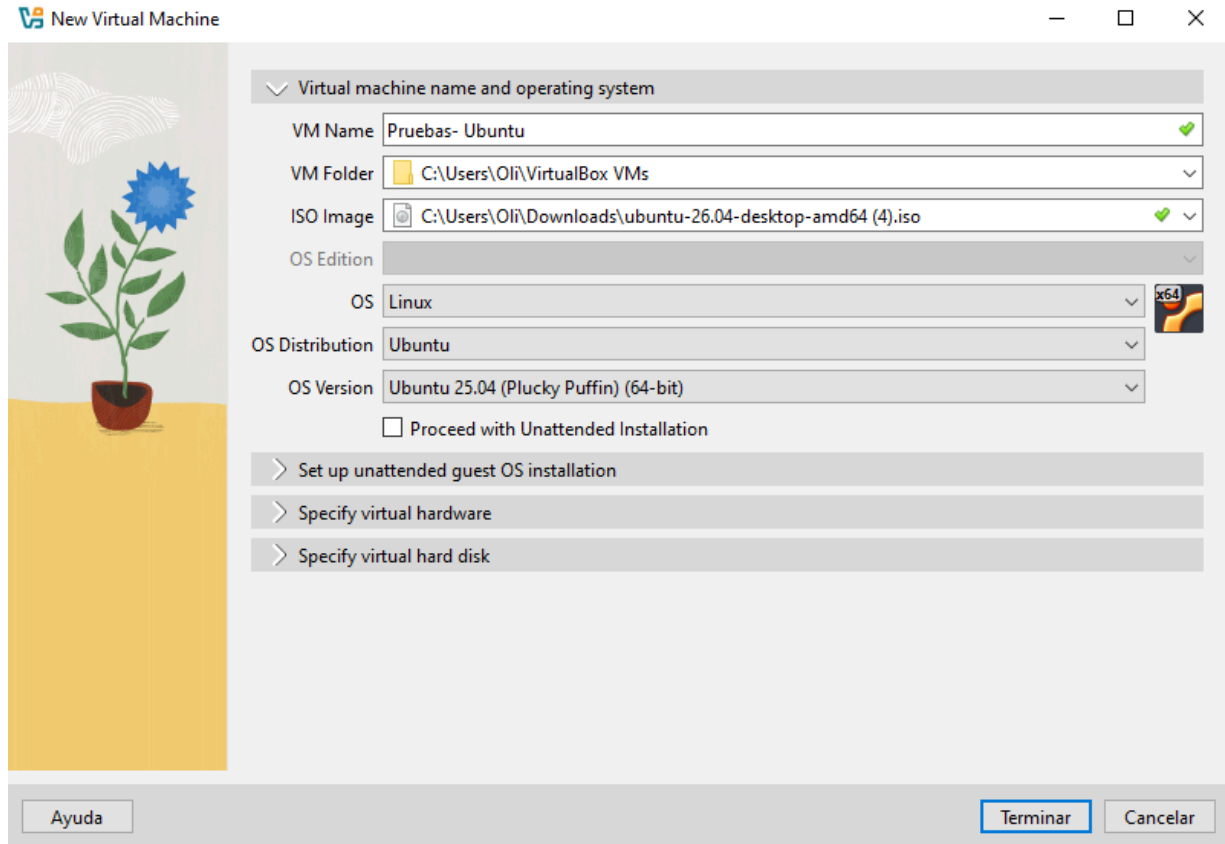


Figura 3. Implementación de máquina virtual. Configuración inicial.

Inmediatamente después de ello, es necesario destinar recursos de Hardware que serán asignados a la VM. es importante destacar que estos recursos de RAM y procesador solo son consumidos mientras la máquina esté en ejecución. Al cerrarla se liberan y vuelven a estar disponibles para el sistema anfitrión. La diferencia a esto se da con el espacio en disco duro asignado, ya que al crear una máquina, se le asigna una determinada cantidad de espacio de memoria, y el programa crea un archivo con extensión .vdi que actúa como un disco duro virtual. Ese archivo no se borra, lo que permite obtener persistencia de datos en las VM. La ventaja es que el archivo ocupa espacio de forma dinámica, es decir que se va llenando a medida que se usa. Si se le asignaron 40 GB por ejemplo, estos no serán ocupados

inmediatamente por la VM, sino que se irán usando a medida que en la máquina se vayan creando archivos, descargando programas, etc.

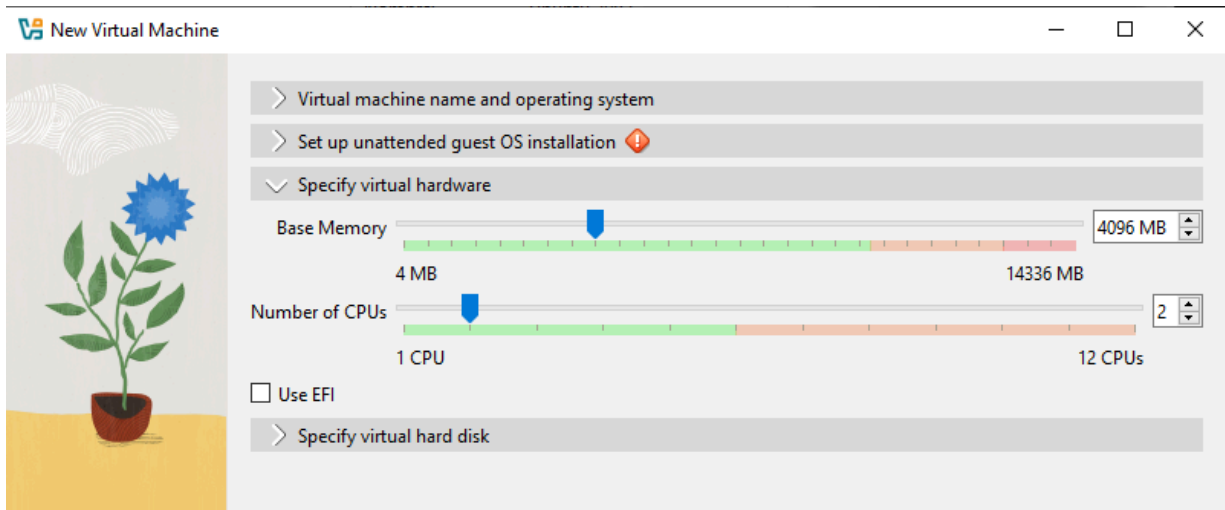


Figura 4. Implementación de máquina virtual. Asignación de memoria RAM y núcleos de CPU

También se debe crear un nombre de usuario y contraseña, que será utilizado al iniciar sesión en el OS invitado, tal como se hace en cualquier otro sistema operativo. Finalmente, se puede optar por instalar algunas extensiones adicionales, y se finaliza la configuración. Una vez terminado esto, se puede iniciar la Máquina virtual, por primera vez, para comenzar la instalación dado que el programa monta el archivo ISO con el instalador del Sistema Operativo.

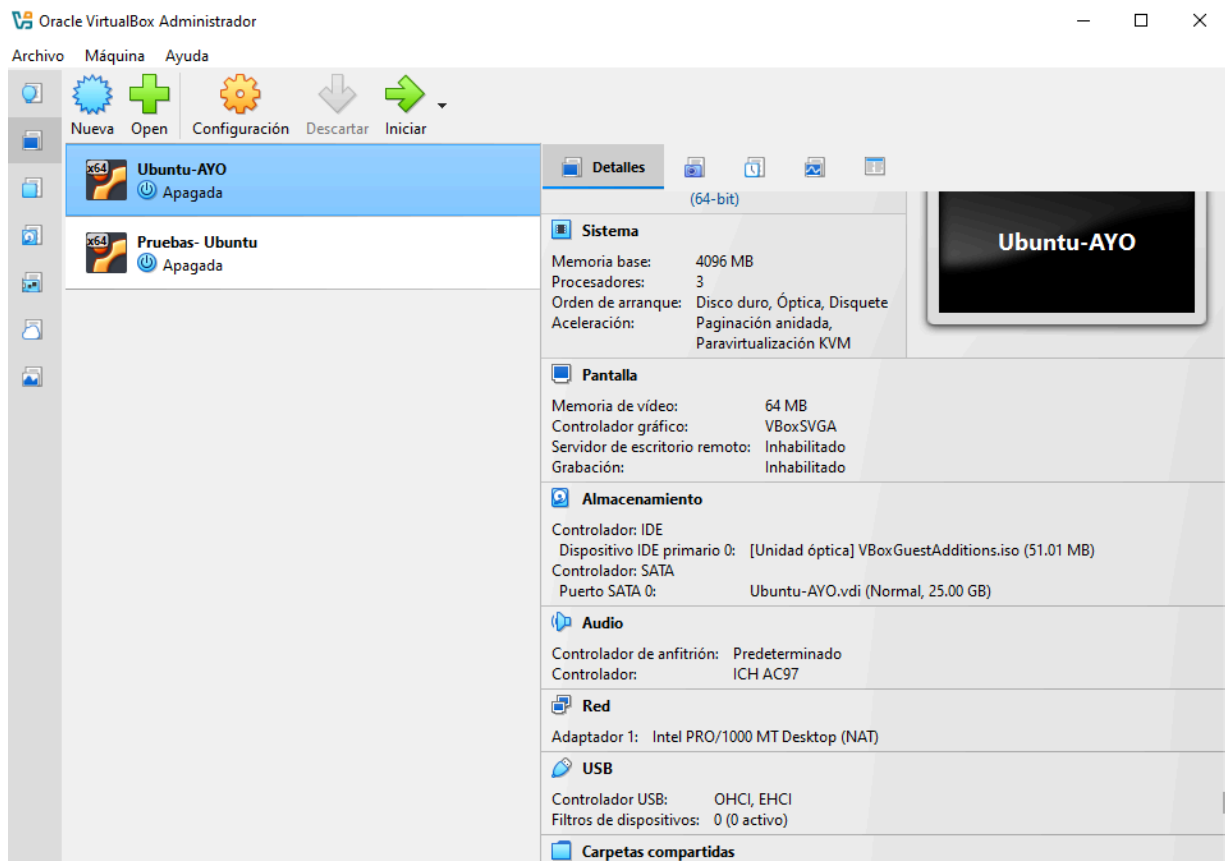


Figura 5. Visión de las máquinas virtuales creadas, con resumen de los recursos asignados

3. Instalando y configurando el Sistema Operativo Ubuntu

Al hacer clic en el botón de iniciar por primera vez la máquina virtual ya creada, el programa inicializará la instalación del sistema operativo configurado. El sistema busca el dispositivo asignado en primer orden para ejecutar, y cómo es un disco , lo ejecuta, conteniendo la instalación del sistema. Se deben seguir los pasos como en la instalación de cualquier sistema operativo. Una vez finalizada la instalación, Ubuntu pide ingresar algunas configuraciones básicas, como el idioma del sistema, el método de ingreso de teclado, el aspecto, etc. Una vez completada la configuración, el sistema está listo para iniciar.

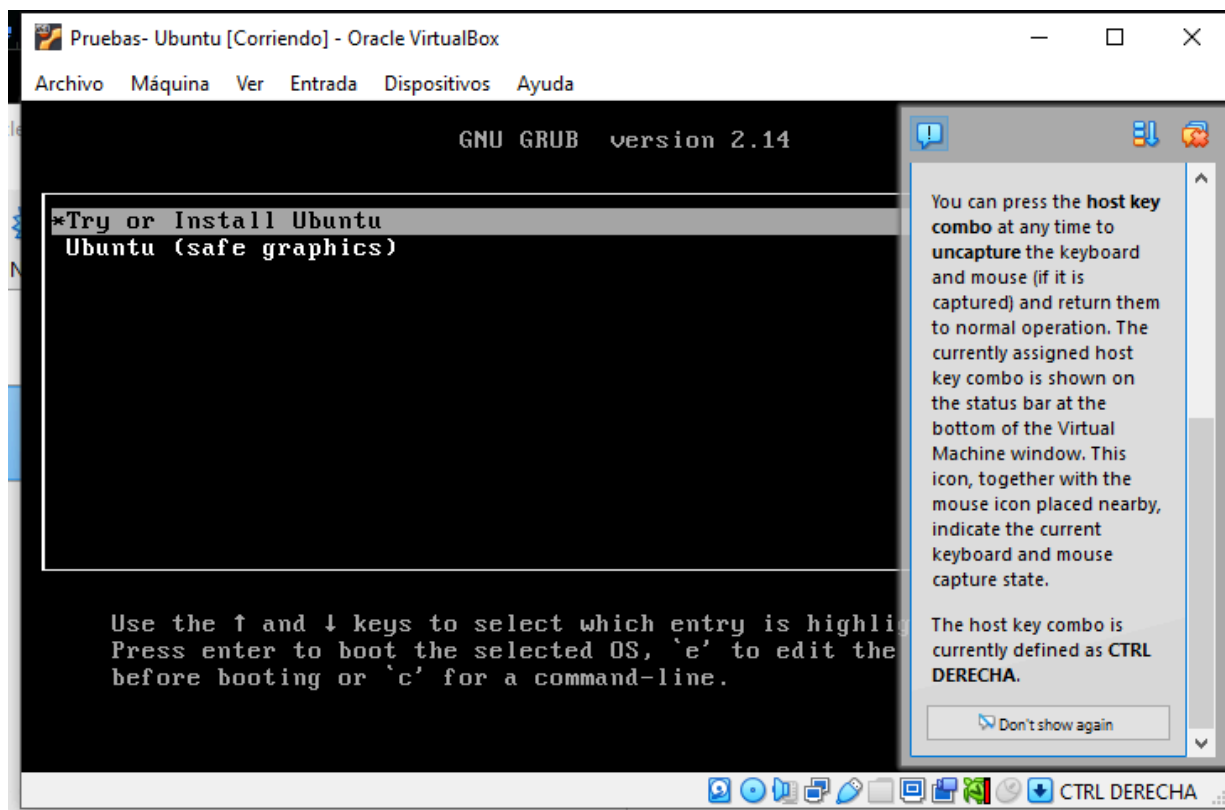


Figura 6. Máquina virtual corriendo y ejecutando el instalador del SO.

4. Creación del código en Python y subida a repositorio remoto.

Paralelamente, se creó un pequeño script de Python por fuera de la máquina virtual. Para ello, se utilizó el IDE Visual Studio Code, y se desarrolló un código simple, cuya estructura fue proporcionada por la cátedra, y se subió al repositorio remoto Github. El objetivo de esto fue implementar código simple, para verificar su funcionalidad en el Sistema Operativo implementado en la Máquina Virtual. Para realizar la carga, se utilizó Github desktop, programa de escritorio que fue recomendado por la cátedra de Programación 1, con el objetivo de subir y sincronizar cambios locales al repositorio. Uno de los miembros del equipo creó y configuró el repositorio en Github, invitó como colaborador al otro miembro, y ambos descargaron una copia local del repositorio para trabajar en sus respectivos equipos. Siguiendo los lineamientos de la

cátedra de Organización Empresarial, los avances fueron subidos a la rama denominada *developer* realizando el *merge* únicamente cuando la versión final estuvo lista.

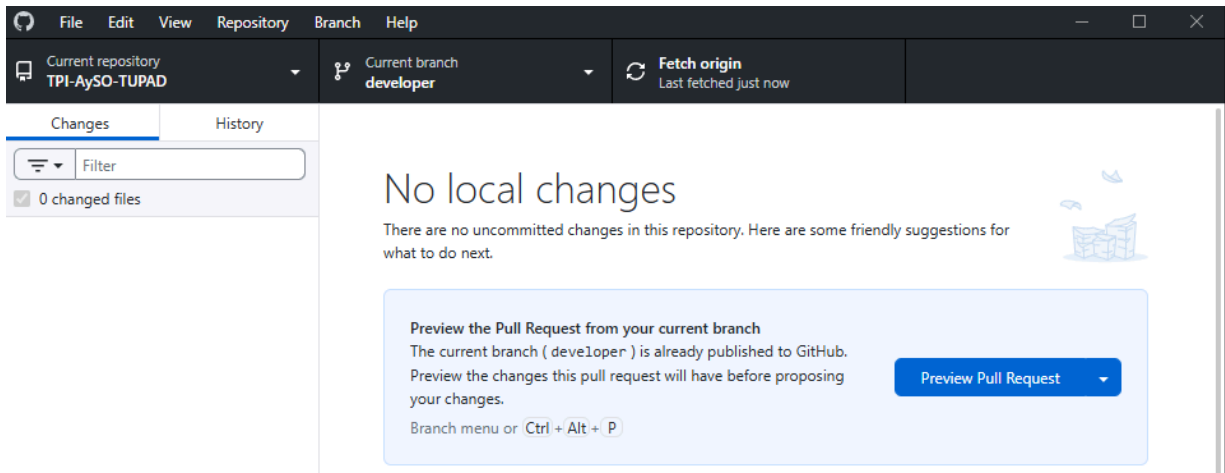


Figura 7. Utilización de la aplicación de escritorio Github Desktop

5. Creación de copia local del repositorio en Ubuntu, y prueba del script de Python.

Finalmente, ya en el sistema implementado en la máquina virtual, se utilizó la terminal para, mediante los comandos de texto, clonar el repositorio remoto, para obtener el script de prueba en Python y ejecutarlo. Para esto, primero fue necesario verificar que Python 3 haya estado correctamente instalado en Ubuntu con el comando `python3 --version`. Una vez hecho esto, se utilizaron los comandos de git para clonar el repositorio.


```
oliverio-arce@oliverio-arce-VirtualBox:~$ git clone https://github.com/oliverio97/TPI-AySO-TUPAD.git
Clonando en 'TPI-AySO-TUPAD'...
remote: Enumerating objects: 6, done.
remote: Counting objects: 100% (6/6), done.
remote: Compressing objects: 100% (5/5), done.
remote: Total 6 (delta 0), reused 3 (delta 0), pack-reused 0 (from 0)
Recibiendo objetos: 100% (6/6), listo.
oliverio-arce@oliverio-arce-VirtualBox:~$ cd TPI-AySO-TUPAD
oliverio-arce@oliverio-arce-VirtualBox:~/TPI-AySO-TUPAD$ git checkout developer
rama 'developer' configurada para rastrear 'origin/developer'.
Cambiado a nueva rama 'developer'
```

Figura 8. Creación de copia local del repositorio.

Finalmente, al obtener la copia local del repositorio, esto también incluía el archivo de Python desarrollado. El mismo pide tres notas al usuario a ingresar por consola, y calcula el promedio de las mismas, mostrando el resultado por pantalla. Se ejecutó el programa mediante la misma terminal de comandos, llamando al nombre del archivo.

```
oliverio-arce@oliverio-arce-VirtualBox:~/TPI-AySO-TUPAD$ python3 promedio.py
Bienvenido al programa para calcular promedios. Ingresá 3 notas para calcular el promedio de las mismas
Ingresa la nota 1: 7.5
Ingresa la nota 2: 8.5
Ingresa la nota 3: 6
El promedio entre las notas es: 7.333333333333333
oliverio-arce@oliverio-arce-VirtualBox:~/TPI-AySO-TUPAD$
```

Figura 9. Prueba del correcto funcionamiento del Script desarrollado.

METODOLOGÍA UTILIZADA

El desarrollo del presente Trabajo Integrador se llevó a cabo mediante un enfoque estructurado, iterativo y colaborativo, dividido en cuatro fases consecutivas: investigación preliminar, planificación y diseño, implementación técnica con herramientas específicas, y distribución equitativa del esfuerzo de trabajo dentro del equipo. A continuación, se detallan los procedimientos, criterios y recursos adoptados en cada una de estas etapas.

Investigación Previa y Fuentes de Información

Antes de proceder con la configuración de los entornos virtuales o la escritura de código, se realizó una exhaustiva fase de recopilación y validación de información bibliográfica. El objetivo principal consistió en cimentar una base sólida de conocimientos que permitiera mitigar errores de compatibilidad y optimizar el rendimiento de la simulación.

Las fuentes consultadas se clasificaron de la siguiente manera:

- Documentación Oficial de Oracle VirtualBox: Se analizaron los manuales de usuario del hipervisor para comprender la gestión de recursos de hardware (memoria RAM, núcleos de procesamiento y almacenamiento dinámico en disco duro).
- Repositorios de Soporte de Ubuntu Linux: Se consultó la documentación oficial de la distribución de Ubuntu con el fin de asimilar la estructura del sistema de archivos, el manejo de permisos mediante la terminal de comandos y los mecanismos de actualización de repositorios nativos.
- Documentación de Referencia de Python: Se revisaron las especificaciones de la versión estable de Python 3, sus librerías estándar y las directrices de sintaxis necesarias para garantizar la correcta portabilidad y ejecución del script en entornos basados en Linux.
- Apuntes y Material de Cátedra: Los módulos teóricos de la materia *Arquitectura y Sistemas Operativos* proveyeron el marco académico para articular los conceptos de virtualización, administración de procesos y comunicación entre el sistema anfitrión (host) y el sistema invitado (guest).

Etapas de Diseño y Prueba del Código

La fase de diseño se estructuró de manera previa a cualquier tipo de implementación en las computadoras de trabajo. En primera instancia, se elaboró un esquema lógico o mapa de flujo que determinó los pasos exactos requeridos para cumplir con los objetivos del proyecto.

Este diseño previo contempló la siguiente secuencia de tareas:

1. Esquematización del Entorno: Definición de la arquitectura de la máquina virtual (asignación óptima de memoria y disco) y selección de la configuración más eficiente para permitir el despliegue del script.
2. Modelado del Script de Python: Diseño algorítmico del archivo de código fuente en el sistema anfitrión, asegurando que las funciones implementadas respondieran a las consignas solicitadas sin requerir dependencias externas complejas.
3. Protocolo de Transferencia: Planificación del puente de comunicación para mover el código desde el sistema operativo base hacia la máquina virtualizada, anticipando posibles barreras de seguridad o restricciones en los portapapeles de los sistemas operativos. Esto fue realizado mediante la utilización del repositorio remoto, el cual fue clonado en el sistema virtual.
4. Pruebas (Testing): Diseño de un conjunto de pruebas básicas para verificar el comportamiento del código una vez transferido. Se realizaron múltiples ejecuciones del programa para validar su correcto funcionamiento, introduciendo datos incorrectos como caracteres en lugar de números, deteniendo la ejecución del programa, etc.

Herramientas y Recursos Utilizados

Para garantizar un estándar profesional de desarrollo, se seleccionó un conjunto de herramientas de software que facilitaron tanto la edición de código como la administración de las versiones del proyecto.

- Entorno de Desarrollo Integrado (IDE) - Visual Studio Code: Se seleccionó esta herramienta en el sistema anfitrión para la escritura del script de Python debido a su flexibilidad, el resaltado de sintaxis, la integración de terminales y la disponibilidad de extensiones de depuración que optimizaron los tiempos de codificación.
- Sistema de Control de Versiones - Git y GitHub: La gestión de los archivos del proyecto se realizó mediante un repositorio remoto alojado en GitHub. Esto permitió mantener un

historial cronológico de cambios, centralizar el código fuente y resguardar el trabajo ante posibles pérdidas de datos locales.

- Interfaz Gráfica de Control - GitHub Desktop: Con el propósito de agilizar los procesos de confirmación de cambios (*commits*), sincronización (*push*) y ramificación de código, se utilizó el cliente oficial GitHub Desktop en el sistema anfitrión, permitiendo una visualización clara del flujo de trabajo y de las ramas de desarrollo (como la rama *developer*) antes del despliegue en la máquina virtual.
- Entorno de Virtualización - Oracle VirtualBox: Utilizado como el hipervisor de Tipo 2 para dar soporte a la máquina virtual con interfaz gráfica de Ubuntu, configurando las herramientas de integración necesarias para la interacción entre los sistemas.
- Sistema Operativo virtualizado: Se utilizó Ubuntu 26.04 LTS como sistema operativo para montar e instalar en la máquina virtual, al ser este libre y gratuito, y ser liviano y de fácil implementación.

Trabajo Colaborativo y Reparto de Tareas

El desarrollo de este Trabajo Integrador se basa en los principios del trabajo colaborativo, implementando una división de tareas equitativa. Ambos integrantes mantuvieron una comunicación fluida a lo largo de todo el proceso, y si bien se realizó división de tareas, se procuró que los dos desarrolladores lograran una comprensión integral del proyecto en su totalidad. El reparto de tareas se dio de la siguiente manera:

Integrante 1 - Mauro Liciaga: investigación bibliográfica inicial, instalación y configuración base de la máquina virtual en VirtualBox, y redacción preliminar del Marco Teórico.

Integrante 2 - Oliverio Arce: investigación bibliográfica inicial, diseño y codificación del script de Python en el IDE, pruebas de depuración en el sistema anfitrión, desarrollo de caso práctico y metodología, y estructuración inicial del repositorio de Git.

RESULTADOS OBTENIDOS

El proyecto se desarrolló de manera satisfactoria, cumpliendo con todos los objetivos planteados al inicio del trabajo. En primer lugar, se descargó e instaló correctamente VirtualBox, se creó una máquina virtual con la configuración necesaria para ejecutar Ubuntu 26.04 LTS. Luego el sistema operativo fue instalado y configurado sin inconvenientes, verificando su correcto funcionamiento.

En paralelo, se desarrolló un script en Python utilizando Visual Studio Code, que se almacenó en un repositorio de GitHub siguiendo una metodología de trabajo colaborativa. Luego desde la máquina virtual, se comprobó que Python estuviera instalado correctamente y se utilizó Git para clonar el repositorio remoto para obtener una copia local del proyecto.

Por último, se ejecutó el script desde la terminal de Ubuntu ingresando los datos solicitados y obteniendo el promedio esperado sin presentar errores durante su ejecución.

CONCLUSIONES

A partir del desarrollo de este proyecto integrador fue posible aplicar de manera práctica los conceptos trabajados durante la cursada, implementando de manera práctica los conceptos teóricos sobre virtualización y sistemas operativos. La instalación de Virtualbox, la creación de la máquina virtual y la posterior instalación de Ubuntu nos permitieron comprender cómo funciona un sistema operativo y cómo se administran los recursos del equipo anfitrión.

La utilización de GitHub permite reforzar los conocimientos sobre el control de versiones y el trabajo colaborativo, mientras que la ejecución del script de Python dentro de la máquina virtual demostró que el entorno fue funcional para desarrollar y ejecutar aplicaciones.

En relación a las dificultades experimentadas, estas fueron principalmente dadas por la dificultad de correr el hipervisor en las computadoras de los integrantes. El sistema operativo no se instalaba, y buscando en internet sobre posibles causas, se decidió asignar más recursos de memoria RAM y núcleos del procesador, para que la instalación pueda correr más fluidamente.

Para posibles proyectos se proponen mejoras relacionadas con implementar y configurar más eficazmente el hipervisor, con los conocimientos adquiridos en el presente trabajo. Otra posible mejora sería implementar distintos sistemas operativos además de Ubuntu, con el objetivo de probar como funciona cada uno de estos, en especial fue de interés de los integrantes realizar una instalación de algún sistema sin interfaz gráfica, que se maneje exclusivamente por línea de comandos, para poder experimentar y aplicar los conocimientos aprendidos en la unidad de línea de comandos vista en la materia.

En conclusión, el proyecto permitió integrar conceptos de la materia, combinando conocimientos de programación y matemáticas sumado a las herramientas utilizadas. La

experiencia resultó enriquecedora ya que nos permite pasar de los conceptos teóricos a una aplicación práctica fortaleciendo de esta manera la comprensión de los temas abordados.

BIBLIOGRAFÍA

- Canonical. (s.f.). Ubuntu documentation. Recuperado el 22 de junio de 2026, de <https://ubuntu.com/desktop/docs>
- Oracle Corporation. (2026). Oracle® VM VirtualBox®: User Manual. Recuperado el 22 de junio de 2026, de <https://www.virtualbox.org/manual/>
- Python Software Foundation. (2026). Python 3.12 documentation. Recuperado el 22 de junio de 2026, de <https://docs.python.org/3/>
- Universidad Tecnológica Nacional. (2026). Apuntes de cátedra: Arquitectura y Sistemas Operativos. Tecnicatura Universitaria en Programación.

ANEXO

Link al repositorio de Github: <https://github.com/oliverio97/TPI-AySO-TUPAD>

Link al video explicativo: <https://www.youtube.com/watch?v=hDCudoaXqsA>